Why better open-source HEO applications are needed

The gap is not another orbit propagator

Open-source astrodynamics already has credible building blocks. Orekit exposes analytical, numerical, and semi-analytical propagation, detailed force models, maneuvers, event detectors, covariance propagation, and orbit-determination algorithms.[1] Nyx provides a performance-oriented Rust/Python astrodynamics toolkit, while SatKit provides a Python interface for high-precision force-model propagation and covariance/state-transition outputs.[2, 3]

The unmet need is a **validated, HEO-specific application layer** that makes those capabilities consistent, reproducible, and operationally usable. Existing open-source options are primarily libraries or toolkits. They do not, from the retrieved evidence, establish a common HEO benchmark, common force-model configuration, common event-time qualification, or a turnkey workflow for Molniya and GTO operations.

What current applications do not fully provide

1. A common HEO validation contract

The current tools expose capabilities, but the user must still decide how to configure gravity, atmosphere, third bodies, radiation pressure, maneuvers, time scales, event detection, tolerances, and covariance models. Without a shared HEO case manifest and reference ephemerides, two users can run "the same" orbit with materially different assumptions.

This matters because HEO propagation is unusually sensitive to numerical and modeling choices. Sundman transformations improve the allocation of computational effort between perigee and apogee, while KS methods can improve integration behavior for perturbed two-body problems.[4, 5] But event-based stopping can add timing or phasing error even when geometric state accuracy is excellent.[6] A better application should therefore ship qualification cases and enforce event-time as a first-class acceptance metric.

2. HEO-specific adaptive fidelity

A production HEO application should select force-model complexity from an explicit error budget rather than making users choose a fixed gravity degree or a single global tolerance. A Molniya case study demonstrates the value of adjusting geopotential degree and numerical tolerance to the required positional accuracy and altitude-dependent perturbation magnitude.[7]

The missing product behavior is an auditable controller that can say: "this force term is required for the current accuracy target and orbital segment," or "this term can be disabled without exceeding the error budget." Current libraries provide many of the force models, but the retrieved material does not show a turnkey HEO fidelity manager with error-budget reporting.

3. Event and maneuver certification

For GTO and other perigee-intensive missions, the most important output is often not the state at an arbitrary time but the timing and state at perigee, burn start, burn stop, eclipse transition, apogee, or ground-contact boundaries. Existing libraries expose event and maneuver primitives, but a deployable HEO application should certify them against reference event times and report event-time uncertainty separately from state-vector error.

The need is especially acute for regularized propagation, where the geometric trajectory can be highly accurate while reconstructed timing is not.[6] A useful application would make missed events, late events, and event-time sensitivity release-blocking failures rather than hidden warnings.

4. Integrated covariance and orbit-determination workflows

Orekit documents orbit-determination and covariance functionality, and SatKit documents state-transition and covariance propagation.[1, 3] What is still missing at the application level is a HEO-specific estimation workflow that couples tracking geometry, perigee passage, maneuvers, process noise, and covariance validation.

A stronger open-source application would include synthetic and real-data examples for Molniya and GTO arcs, normalized-residual diagnostics, NEES tests, three-sigma coverage tests, maneuver covariance injection, and automatic detection of overconfident covariance. This would turn propagation libraries into navigation evidence rather than merely state generators.

5. Reproducible cross-language deployment

The three open-source options span Java, Rust/Python, and Python-oriented workflows.[1–3] That diversity is valuable, but it increases the need for common data schemas, deterministic reference outputs, version-pinned containers, and conformance tests. A deployable application should provide CCSDS input/output, explicit frame and time-scale metadata, machine-readable force-model configuration, and a test suite that can run identically in CI, on a ground server, and in a flight-software preproduction environment.

6. HEO operations rather than only analysis

Commercial products demonstrate the operational breadth users expect: FreeFlyer describes full mission lifecycle support, configurable force models, orbit determination, covariance, collision-avoidance analysis, and APIs for integration with ground systems.[8] Open-source alternatives do not need to reproduce every commercial feature, but they need a credible path from library call to operational product: job scheduling, ephemeris versioning, reproducible reruns, alerting, event audit trails, and integration tests with tracking and command systems.

What a better open-source application could offer

A high-value product would be an **HEO propagation and navigation workbench** built on existing libraries rather than a new dynamics engine. Its differentiating capabilities would be:

- **Mission templates**: Molniya, Tundra, GTO, lunar HEO, and custom high-eccentricity cases.
- **Error-budget-driven fidelity**: automatic force-model and tolerance selection with a written accuracy budget.
- **Dual propagator mode**: regularized propagation for difficult perigee dynamics and conventional numerical or semi-analytical propagation for long arcs, with cross-checks between them.
- **Certified events**: perigee, apogee, eclipse, ground contact, maneuver, and conjunction events with timing residuals against reference cases.
- **Navigation mode**: measurement simulation, batch least squares, sequential filtering, covariance propagation, maneuver uncertainty, and NEES/coverage diagnostics.
- **Reproducibility**: version-pinned environments, immutable case manifests, CCSDS-compatible files, deterministic seeds, and exportable provenance.
- **Scalable execution**: one spacecraft through large constellations, with an accuracy–runtime Pareto report rather than an opaque "fast mode."
- **Operational interfaces**: REST/Python/Rust/Java APIs, command-line batch operation, container deployment, ephemeris services, and ground-system adapters.
- **Qualification tooling**: automated regression tests that fail on event-time drift, covariance inconsistency, force-model discontinuities, or unexplained changes after a dependency update.

Highest-value first release

The best initial product is not a broad replacement for commercial mission-analysis suites. It is a narrow, authoritative HEO qualification and operations stack:

1. Molniya and GTO case manifests with public initial states and force-model inputs.
2. Independent high-accuracy reference ephemerides.
3. Orekit/Nyx/SatKit adapters implementing one common interface.
4. Automated state, event, runtime, OD, and covariance comparisons.
5. A dashboard showing accuracy, event-time error, covariance consistency, and computational cost.
6. A reproducible command-line runner suitable for continuous integration and ground deployment.

That product would fill a gap the current libraries do not fill alone: it would make HEO propagation **qualified**, **comparable**, **and operationally auditable**. The commercial benchmark is not feature count; it is confidence that a perigee event, maneuver, covariance, or long-arc prediction can be reproduced and defended.

Decision criteria for investment

Development is justified if users need any of the following and are currently assembling them manually: repeated HEO mission studies, open verification of commercial-tool outputs, high-latitude constellation operations, GTO maneuver analysis, independent orbit-determination validation, or a transparent path from research code to operational deployment. The strongest initial users are likely organizations that need auditability and customization but cannot accept a black-box commercial workflow.

The main technical risk is not lack of algorithms. It is integration: agreeing on force-model data, time and frame conventions, event semantics, covariance handling, and release qualification. An open-source application that solves those integration and validation problems would be meaningfully different from another general-purpose propagator API.

References

1. Overview (OREKIT 12.0.2 API). https://test.orekit.org/site-orekit-12.0.2/apidocs/index.html

2. nyx-space/nyx. https://github.com/nyx-space/nyx

3. Propagation - SatKit. https://satkit.dev/api/satprop/

4. Berry M, Healy L (2002) The generalized Sundman transformation for propagation of high-eccentricity elliptical orbits

5. Fukushima T (2004) Efficient Integration of Highly Eccentric Orbits by Scaling Methods Applied to Kustaanheimo-Stiefel Regularization. The Astronomical Journal. https://doi.org/10.1086/425553

6. Urrutxua H, Álvarez J, Bombardelli C (2016) Impact of the integration stop-condition on the accuracy of regularized orbit formulations

7. Flores R, Burhani B, Fantino E (2021) A Method for Accurate and Efficient Propagation of Satellite Orbits: A Case Study for a Molniya Orbit. arXivorg. https://doi.org/10.1016/j.aej.2020.12.056

8. Flight dynamics software for spacecraft mission design, analysis and operations. https://ai-solutions.com/wp-content/uploads/2026/08/a.i.-solutions-FreeFlyer-Brochure-August-2026-FINAL.pdf